ADAPTIVE COMPRESSION ANALYZER USING MULTIPLE ALGORITHMS

A PROJECT REPORT

Submitted by

Manvi Tyagi (24BCS10495)

Under the guidance of

Dr. Sumit Kumar Rana

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE ENGINEERING

Chandigarh University

April 2026

TABLE OF CONTENTS

CHAPTER 1. INTRODUCTION..... 5-6

1.1. Identification of Client/ Need/ Relevant Contemporary issue.....5
1.2. Identification of Problem..... 5
1.3. Identification of Tasks..... 5-6
1.4. Timeline..... 6
1.5. Organization of the Report.....6

CHAPTER 2. LITERATURE REVIEW/BACKGROUND STUDY..... 7-8

2.1. Timeline of the reported problem..... 7
2.2. Existing solutions.....7
2.3. Bibliometric analysis..... 7
2.4. Review Summary.....8
2.5. Problem Definition.....8
2.6. Goals/Objectives.....8

CHAPTER 3. DESIGN FLOW/PROCESS..... 9-11

3.1. Evaluation & Selection of Specifications/Features.....9
3.2. Design Constraints.....9
3.3. Analysis of Features and finalization subject to constraints.....10
3.4. Design Flow.....10
3.5. Design selection.....11
3.6. Implementation plan/methodology..... 11

CHAPTER 4. RESULTS ANALYSIS AND VALIDATION.....12-14

4.1. Implementation of solution.....12-13
4.2. Comparative.....14

CHAPTER 5. CONCLUSION AND FUTURE WORK..... 15

5.1. Conclusion..... 15
5.2. Future work.....15

REFERENCES..... 16

ABSTRACT

Data compression is a fundamental concept in computer science that focuses on reducing the size of data to improve storage efficiency and transmission speed. With the increasing volume of digital data, selecting an appropriate compression technique has become an important challenge. Different algorithms perform differently depending on the characteristics of the input data, such as repetition patterns and frequency distribution.

This project presents an Adaptive Compression Analyzer that evaluates the performance of three compression techniques: Fixed-Length Encoding, Run Length Encoding (RLE), and Huffman Encoding. The system takes input text and calculates the size of compressed data in terms of bits for each algorithm. The implementation focuses on analyzing algorithmic efficiency rather than performing real-world file compression.

Fixed-Length Encoding is used as a baseline method where each character is represented using a fixed number of bits. Run Length Encoding is applied to compress repetitive data by storing characters along with their frequency of occurrence. Huffman Encoding, based on a greedy approach, assigns variable-length codes to characters depending on their frequency, resulting in more efficient compression for uneven data distributions.

The analyzer compares the bit usage of all three algorithms and identifies the most efficient one for the given input. The results demonstrate that no single compression algorithm performs best for all types of data. Instead, the efficiency depends on the structure and pattern of the input.

This project highlights the importance of algorithm selection in data compression and provides a clear understanding of how different techniques behave under varying data conditions.

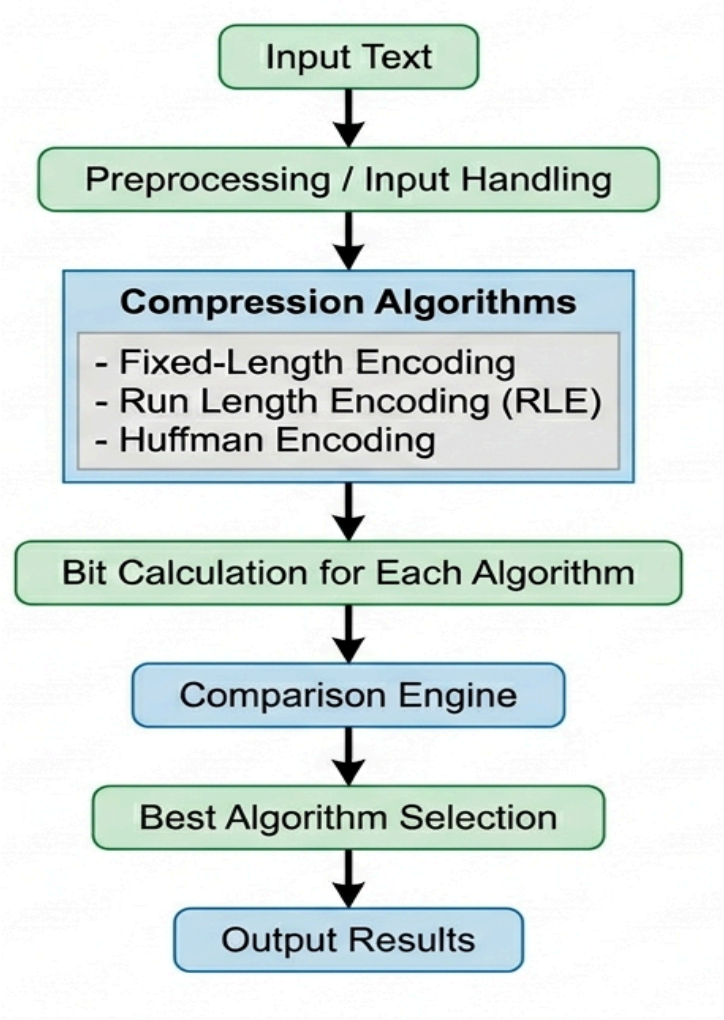
GRAPHICAL ABSTRACT

The graphical abstract represents the working of the Adaptive Compression Analyzer system. The process begins with user input in the form of text data. The input is then passed through multiple compression algorithms, namely Fixed-Length Encoding, Run Length Encoding (RLE), and Huffman Encoding.

Each algorithm processes the same input and calculates the size of the compressed data in terms of bits. These results are forwarded to a comparison engine, which evaluates the performance of each algorithm.

Based on the minimum number of bits required, the system selects the most efficient compression technique for the given input. The final output displays the compressed size for each algorithm along with the best-performing method.

This diagram illustrates the overall workflow of the system and highlights how different algorithms are applied and compared to achieve adaptive compression analysis.



INTRODUCTION

1.1. Identification of Client /Need / Relevant Contemporary issue

In the modern digital era, the amount of data being generated and processed has increased significantly. Efficient storage and transmission of data have become critical concerns for individuals, organizations, and industries. Data compression plays a key role in addressing these challenges by reducing the size of data without losing essential information.

Various compression techniques are available, but their performance varies depending on the nature of the input data. For example, repetitive data benefits from certain algorithms, while unevenly distributed data requires different approaches. According to reports on data growth and storage demands, efficient data handling techniques are essential for optimizing system performance and reducing storage costs.

Currently, most systems apply a single compression algorithm without analyzing whether it is the most efficient for the given data. This creates a gap where users or systems lack the ability to evaluate and select the best compression technique dynamically.

Therefore, there is a need for a system that can analyze input data and determine the most efficient compression algorithm based on its characteristics. This project addresses this need by developing an Adaptive Compression Analyzer that compares multiple algorithms and identifies the optimal one.

1.2. Identification of Problem

The broad problem addressed in this project is the absence of a systematic approach to evaluate and compare different data compression algorithms for a given input.

Different compression algorithms exhibit varying performance depending on data characteristics such as repetition and frequency distribution. However, there is no simple analytical system that allows users to measure and compare the efficiency of multiple algorithms on the same input data.

As a result, it becomes difficult to determine which compression technique is most suitable under specific conditions. The problem is to design a system that can evaluate multiple algorithms and provide a clear comparison based on performance metrics such as bit usage.

1.3. Identification of Tasks

The project is divided into a series of tasks required to design, implement, and evaluate the solution.

- Study of Data Compression Concepts:
  Understanding the principles of encoding, bit representation, and compression techniques.

- Implementation of Fixed-Length Encoding:
  Developing a baseline method to calculate data size using constant bit representation.

- Implementation of Run Length Encoding (RLE):
  Designing a method to compress repetitive data using character-count representation.
- Implementation of Huffman Encoding:
  Applying a greedy algorithm approach to compress data based on frequency distribution.
- Development of Comparison Mechanism:
  Creating logic to compare the number of bits used by each algorithm.
- Testing and Validation:
  Running the system on different types of input data such as repetitive, random, and skewed data.
- Result Analysis:
  Evaluating which algorithm performs best under different conditions.

These tasks collectively form the framework of the project and are further detailed in subsequent chapters.

1.4. Timeline

The project was completed over a period of six weeks with contributions from all three group members. The work was divided into different phases to ensure systematic development.

- Week 1: Requirement analysis and study of compression algorithms
- Week 2: Implementation of Fixed-Length Encoding
- Week 3: Implementation of Run Length Encoding
- Week 4: Implementation of Huffman Encoding
- Week 5: Integration of algorithms and development of comparison logic
- Week 6: Testing, result analysis, and report preparation

1.5. Organization of the Report

This report is organized into five chapters.

Chapter 1 introduces the problem, need, and objectives of the project.
Chapter 2 covers the literature review and background of compression techniques.
Chapter 3 explains the design, methodology, and implementation details.
Chapter 4 presents the results and analysis of the system.
Chapter 5 concludes the project and suggests future improvements.

The report also includes references and appendices for additional information.

LITERATURE REVIEW/BACKGROUND STUDY

2.1. Timeline of the reported problem

The need for data compression has existed since the early days of computing, when storage and transmission capacity were limited. Early techniques like ASCII used fixed-length encoding, while later methods such as Run Length Encoding (RLE) and Huffman Encoding provided more efficient compression based on data patterns and frequency.

Although these algorithms are widely used and studied, students often learn them only theoretically and lack tools to analyze their performance on different types of data. Modern educational approaches emphasize practical understanding, creating a need for systems that demonstrate how compression algorithms work and how their efficiency varies with input characteristics.

2.2. Existing solutions

Various compression algorithms have been developed to reduce data size, each with its own advantages and limitations. Fixed-Length Encoding uses a constant number of bits per character and does not achieve compression. Run Length Encoding (RLE) is effective for repetitive data but performs poorly on random data. Huffman Encoding uses variable-length codes based on frequency and provides efficient compression for uneven data distributions.

Most existing implementations focus on individual algorithms rather than comparing them on the same input. Additionally, many tools are complex and not designed for educational purposes.

2.3. Bibliometric analysis

The analysis of existing compression techniques is based on key features, effectiveness, and limitations.

Fixed-Length Encoding:
Feature – Simple and uniform representation
Effectiveness – No compression achieved
Drawback – Inefficient for all data types

Run Length Encoding:
Feature – Compresses repeated characters
Effectiveness – Highly efficient for repetitive data
Drawback – Inefficient for non-repetitive data

Huffman Encoding:
Feature – Variable-length encoding based on frequency
Effectiveness – High compression efficiency for uneven data
Drawback – More complex compared to basic methods

2.4. Review Summary

The literature review indicates that multiple compression algorithms exist, each designed for specific data patterns. However, no single algorithm provides optimal performance for all types of input data.

Existing studies primarily focus on algorithm implementation rather than comparative analysis. There is a lack of simple and interactive tools that allow learners to observe and analyze the behavior of different compression techniques on the same input.

This project addresses this gap by providing a system that compares multiple algorithms and identifies the most efficient one based on measurable parameters such as bit usage.

2.5. Problem Definition

The problem addressed in this project is to design a system that can analyze and compare multiple compression algorithms to determine the most efficient one for a given input.

The system will take input text, apply different compression algorithms, calculate the number of bits used, and identify the best-performing algorithm.

The project focuses on algorithmic analysis and educational understanding rather than implementing real-world file compression systems. It does not involve advanced features such as file handling, binary streams, or large-scale data processing.

2.6. Goals/Objectives

The objectives of the project are:

- To understand the concept of data compression and encoding
- To implement Fixed-Length Encoding, Run Length Encoding, and Huffman Encoding
- To calculate and compare the number of bits used by each algorithm
- To analyze how different algorithms perform on different types of input data
- To identify the most efficient compression technique for a given input
- To provide a clear and practical understanding of compression algorithms for learners

These objectives are specific, measurable, and can be validated through testing and analysis.

DESIGN FLOW/PROCESS

3.1. Evaluation & Selection of Specifications/Features

Based on the literature review, key features for the system were identified to analyze and compare different compression algorithms on the same input. These include user input handling, implementation of multiple algorithms, bit calculation, performance comparison, and selection of the most efficient algorithm.

The system is designed to provide simple and clear output for educational understanding. Features like complex file handling and real-time compression are excluded to maintain simplicity and focus on algorithm analysis.

3.2. Design Constraints

The design of the system is subject to the following constraints:

Standards:
The system follows basic data encoding standards such as ASCII representation (8-bit per character).

Economic Constraints:
The project is designed to be low-cost and implemented using standard tools such as C++ and Visual Studio Code.

Technical Constraints:
The implementation avoids complex file handling and focuses only on logical bit calculation for simplicity.

Educational Constraints:
The system is designed for learners, so clarity and simplicity are prioritized over optimization.

Ethical and Social Considerations:
The project is intended for educational use and does not involve sensitive data or harmful applications.

Time Constraints:
The project is completed within a limited academic timeline of six weeks.
These constraints ensure that the system remains simple, understandable, and aligned with DAA concepts.

3.3. Analysis of Features and finalization subject to constraints

Based on the identified constraints, the features were refined to ensure feasibility and clarity.

Retained Features:

- Implementation of Fixed-Length Encoding, RLE, and Huffman Encoding
- Bit calculation for each algorithm
- Comparison and selection of best algorithm

Modified Features:

- Instead of actual file compression, logical bit calculation is used
- Input is limited to simple text instead of large datasets

Removed Features:

- File handling and binary compression
- Complex graphical interfaces

The final system focuses on algorithm analysis rather than real-world compression, making it suitable for educational purposes.

3.4. Design Flow

Design 1: Sequential Execution Model

Input → Fixed Encoding → RLE → Huffman → Compare → Output

In this design, the input is processed sequentially through all algorithms. Each algorithm computes the number of bits independently, and the results are compared to determine the best algorithm.

Design 2: Modular Analyzer Model

Input

↓

Algorithm Modules (Fixed / RLE / Huffman)

↓

Bit Calculation

↓

Comparison Engine

↓

Best Algorithm Output

In this design, each algorithm is treated as a separate module. The results are collected and passed to a comparison engine, which selects the best algorithm. This design is more organized and scalable.

3.5. Design selection

Between the two designs, the Modular Analyzer Model is selected.

The sequential model is simple but less flexible, as all operations are tightly connected. In contrast, the modular design separates each algorithm into independent components, making the system easier to manage, understand, and extend.

The modular design also supports future enhancements, such as adding new algorithms without modifying the entire system. Therefore, it is more suitable for an analytical and educational tool.

3.6. Implementation plan/methodology

INPUT: Text string S
OUTPUT: Best compression algorithm with minimum bits

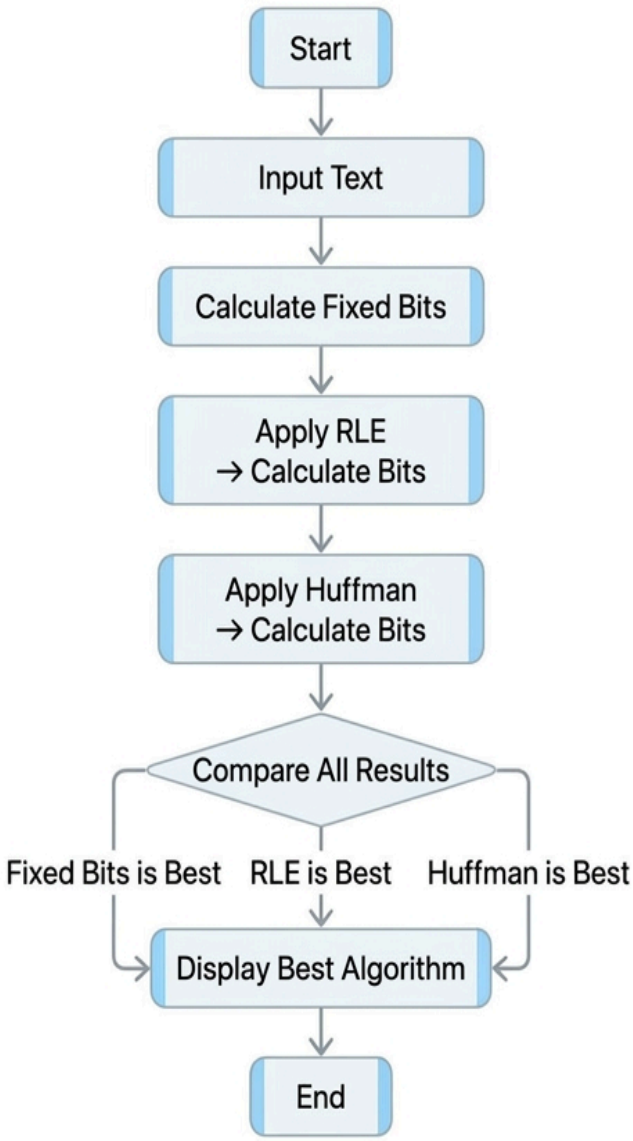
STEP 1: INPUT & BASELINE
```
Read string S, compute length n
FixedBits = n × 8
```

STEP 2: RUN LENGTH ENCODING (RLE)
```
Initialize count = 1, RLEBits = 0
Traverse S:
    If current == next → count++
    Else:
        store (char, count)
        RLEBits += 16
        reset count = 1
```

STEP 3: HUFFMAN ENCODING
```
Build frequency map
Insert into Min Heap
While heap size > 1:
    remove two smallest
    merge and add to HuffmanBits
    insert back
```

STEP 4: COMPARISON & OUTPUT
```
Compare FixedBits, RLEBits, HuffmanBits
Select minimum and display results
```

END



RESULTS ANALYSIS AND VALIDATION

4.1. Implementation of solution

4.1.1 Development Environment

The project was implemented using C++ programming language. Visual Studio Code (VS Code) was used as the development environment. The system was compiled using a standard C++ compiler (g++).

The project is designed as a console-based application for simplicity and ease of understanding.

4.1.2 Project Structure

The project is organized into multiple files for modularity and clarity:

- main.cpp – Handles user input, calls all algorithms, and displays results
- fixed.cpp – Implements Fixed-Length Encoding
- rle.cpp – Implements Run Length Encoding (RLE)
- huffman.cpp – Implements Huffman Encoding

This modular structure improves readability, maintainability, and scalability of the system.

4.1.3 Implementation Logic

The system follows a modular approach where each compression algorithm is implemented independently. The main file takes input from the user and passes it to each algorithm module.

Each algorithm calculates the number of bits required to represent the input data. The results are then compared to determine the most efficient algorithm based on minimum bit usage.

This approach ensures separation of concerns and allows easy extension of the system by adding new algorithms.

4.1.4 Code Snippets

- **Frequency calculation using hash maps in Huffman Encoding**

```
unordered_map<char, int> freq;
for (int i = 0; i < input.length(); i++)
{
   char ch = input[i];
   freq[ch] = freq[ch] + 1;
}
```

A hash map stores character frequencies from the input string, which are then used to build a Huffman tree using a min-heap for efficient encoding.

- **Use of priority queue (min-heap) for building Huffman tree**

```
priority_queue<int, vector<int>, greater<int> > heap;
for (auto it = freq.begin(); it != freq.end(); it++)
{
   heap.push(it->second);
}
```

Min-heap merges smallest frequencies to build the Huffman tree and reduce bits.

- **Loop-based traversal for Run Length Encoding**

```
for (int i = 0; i < n; i++)
{
   int count = 1;
   while (i < n - 1 && input[i] == input[i + 1])
   {
      count++;
      i++;
   }
   encoded += input[i];
   encoded += to_string(count);
   totalBits += 16;
}
```

Counts repeats in a loop and stores char + count to compress data.

- **Bit calculation logic for all algorithms**

```
int fixedBits = CalculateFixedSize(input);
RLEResult rleResult = RunLengthEncode(input);
int huffmanBits = HuffmanEncoding(input);
int minBits = fixedBits;
string bestAlgo = "Fixed Encoding";
if (rleResult.totalBits < minBits)
{
minBits = rleResult.totalBits;
   bestAlgo = "Run Length Encoding (RLE)";
}
if (huffmanBits < minBits)
{ minBits = huffmanBits;
   bestAlgo = "Huffman Encoding"; }
```

It compares bit usage across algorithms and selects the one with the lowest bits as most efficient.
(Refer to Appendix for complete source code)

4.2. Comparative Result Analysis

4.2.1 Repetitive Data Results

```
Enter text to analyze: AAAAAAAABBBBBBBBCCCCCCCCDDDDDDDD

===== Compression Results =====
Fixed Encoding Bits : 256
RLE Encoded Output  : A8B8C8D8
RLE Bits Used       : 64
Huffman Bits Used   : 64

>>> Best Algorithm: Run Length Encoding (RLE)
>>> Minimum Bits Used: 64
```

4.2.2 Random Data Results

```
Enter text to analyze: QWERTYUIOPASDF

===== Compression Results =====
Fixed Encoding Bits : 112
RLE Encoded Output  : Q1W1E1R1T1Y1U1I1O1P1A1S1D1F1
RLE Bits Used       : 224
Huffman Bits Used   : 54

>>> Best Algorithm: Huffman Encoding
>>> Minimum Bits Used: 54
```

4.2.3 Skewed Data Results

```
Enter text to analyze: AAAAAABCDABAAAAAA

===== Compression Results =====
Fixed Encoding Bits : 136
RLE Encoded Output  : A6B1C1D1A1B1A6
RLE Bits Used       : 112
Huffman Bits Used   : 23

>>> Best Algorithm: Huffman Encoding
>>> Minimum Bits Used: 23
```

The results confirm that RLE performs best on repetitive data, while Huffman is more effective for irregular distributions. Fixed-length encoding remains inefficient across all cases due to lack of adaptability.

CONCLUSION AND FUTURE WORK

5.1. Conclusion

The developed system achieves the objective of analyzing and comparing multiple data compression techniques by evaluating their bit consumption for a given input. It provides a structured way to observe how different encoding strategies behave under varying data patterns and helps in identifying the most efficient approach for a specific case.

The results confirm that the effectiveness of a compression technique is highly dependent on the structure of the input data. The system is able to correctly distinguish between cases where repetition-based encoding is beneficial and cases where frequency-based encoding performs better. The selection logic consistently produces a valid best-fit algorithm based on computed bit usage.

Some variation between theoretical expectations and observed outputs exists due to simplified modeling of compression behavior and the use of approximate bit estimation instead of low-level encoding representation. Even with these limitations, the system successfully demonstrates comparative analysis in a controlled and educational manner.

5.2. Future work

The next phase of this project will focus on transforming the system from a static analyzer into an intelligent and data-driven compression decision system.

A key enhancement will be the integration of LangChain-based intelligence, enabling the system to not only compute compression efficiency but also explain and justify why a particular algorithm (Fixed-Length, RLE, or Huffman) is selected for a specific type of input. This will introduce a reasoning layer that connects input characteristics with algorithm behavior in a more interpretable way.

In addition, the system will be extended to support large file processing instead of only user-entered strings, allowing real-world datasets such as text documents to be analyzed. Along with this, a history storage mechanism will be introduced to maintain records of previous inputs, selected algorithms, and compression outcomes. This will enable trend analysis and repeated comparison over time.

Future improvements may also include expanding support to additional compression techniques and enhancing scalability for larger datasets. The overall goal is to evolve the current model into an adaptive, learning-based compression recommendation system that combines algorithmic analysis with contextual intelligence and historical learning.

REFERENCES

GeeksforGeeks

Data Compression, Huffman Coding, Run Length Encoding, and related algorithms
https://www.geeksforgeeks.org/electronics-engineering/what-are-data-compression-techniques/
https://www.geeksforgeeks.org/huffman-coding-greedy-algo-3/
https://www.geeksforgeeks.org/run-length-encoding/

W3Schools

C++ Programming Concepts used for implementation (loops, strings, data structures)
https://www.w3schools.com/cpp/

Programiz

Huffman Coding and basic algorithm explanations
https://www.programiz.com/dsa/huffman-coding